

Corollary 8.2 (A cubic sparse-output lower bound). *There is a sequence of cyclically reduced words w_m with $\|w_m\|_{\mathcal{A}} = 4m$ such that the binary bit-size of the sparse expanded output for f_{w_m} is $\Omega(m^3)$. Consequently, any deterministic algorithm which outputs the fully expanded polynomial in sparse binary form has worst-case running time $\Omega(n^3)$ on inputs of length n .*

Proof. We use the words $w_m = a^m b^m (ab)^m$ from [Theorem 8.1](#), with $n = 4m = |w_m| = |w_m|_{\mathcal{A}} = \|w_m\|_{\mathcal{A}}$. It is enough to look at the coefficient slice of z^{m+1} , namely $E_m(x)E_m(y)$.

Let

$$E_m(t) = \sum_j e_j t^{m-1-2j}.$$

We first note that the sum of the binary lengths of the nonzero coefficients e_j is $\Omega(m^2)$. Indeed, for all integers j in the interval

$$\left\lceil \frac{m}{5} \right\rceil \leq j \leq \left\lfloor \frac{m}{4} \right\rfloor$$

and for all sufficiently large m , the coefficient

$$|e_j| = \binom{m-1-j}{j}$$

is at least 2^{cm} for some absolute constant $c > 0$. For instance, after replacing the displayed interval by a slightly smaller subinterval if necessary, the elementary bound

$$\binom{N}{j} \geq \left(\frac{N-j+1}{j} \right)^j$$

with $N = m-1-j$ gives a lower bound 2^{cm} . There are $\Omega(m)$ such values of j , so

$$\sum_{e_j \neq 0} \log_2(|e_j| + 1) = \Omega(m^2).$$

The nonzero coefficients of $E_m(x)E_m(y)$ are the products $e_i e_j$, with no collisions because the monomials have distinct (x, y) -exponents. Hence the total binary size of this coefficient slice is bounded below by

$$\sum_{i,j} \log_2(|e_i e_j| + 1) \geq \sum_{i,j} (\log_2(|e_i| + 1) + \log_2(|e_j| + 1) - 1).$$

Since E_m has $\Omega(m)$ nonzero coefficients and $\sum_j \log_2(|e_j| + 1) = \Omega(m^2)$, the right-hand side is $\Omega(m^3)$. Therefore the sparse expanded output for f_{w_m} has at least $\Omega(m^3)$ bits. Since $\|w_m\|_{\mathcal{A}} = 4m$, this is $\Omega(n^3)$ in terms of input length n . Any algorithm that actually writes this output must use at least that much time in the worst case. $\square$

Remark 8.3. The preceding lower bound does not match the universal upper bound $O(n^4)$ on dense expanded output size. It leaves open whether the true worst-case sparse output size is cubic, quartic, or somewhere in between. It also leaves open whether the support size can actually be cubic in the word length. Computational evidence does suggest that families such as powers of the commutator may have cubic support.

9. POLYNOMIAL-TIME COMPUTATION IN EXPANDED FORM

The recursive trace identities of [Section 3](#) are conceptually useful, but a more direct way to obtain a polynomial-time bound for the fully expanded polynomial is to evaluate the word on generic matrices over a fixed quadratic extension of $\mathbb{Z}[x, y, z]$. This section proves the elementary complexity statement announced in the introduction.

Let

$$R = \mathbb{Z}[x, y, z, \zeta] / (\zeta^2 - z\zeta + 1).$$

Note that $\mathbb{Z}[x, y, z]$ is naturally embedded in R .

Every element of R has a unique representative of the form

$$P(x, y, z) + Q(x, y, z)\zeta,$$

with $P, Q \in \mathbb{Z}[x, y, z]$.

Define matrices over R by

$$\mathfrak{A} = \begin{pmatrix} x & -1 \\ 1 & 0 \end{pmatrix}, \quad \mathfrak{B} = \begin{pmatrix} 0 & \zeta \\ \zeta - z & y \end{pmatrix}.$$

Lemma 9.1. *We have*

$$\mathfrak{A}, \mathfrak{B} \in SL(2, R)$$

Proof. Indeed,

$$\det \mathfrak{A} = 1, \quad \det \mathfrak{B} = -(\zeta)(\zeta - z) = -\zeta^2 + z\zeta = 1,$$

where the last equality uses the defining relation $\zeta^2 = z\zeta - 1$ in R . $\square$

Definition 9.2. For a word $w = s_1 \cdots s_n$ in the alphabet $a^{\pm 1}, b^{\pm 1}$, set

$$M(a) = \mathfrak{A}, \quad M(a^{-1}) = \mathfrak{A}^{-1} = \begin{pmatrix} 0 & 1 \\ -1 & x \end{pmatrix}, \quad M(b) = \mathfrak{B}, \quad M(b^{-1}) = \mathfrak{B}^{-1} = \begin{pmatrix} y & -\zeta \\ z - \zeta & 0 \end{pmatrix},$$

and define

$$M(w) := M(s_1) \cdots M(s_n) \in M_2(R).$$

Proposition 9.3 (A quadratic generic matrix model). *For every freely reduced word $w \in F(a, b)$ we have*

$$\text{Tr } M(w) = f_w(x, y, z)$$

as an element of R . In particular, if the unique representative of $\text{Tr } M(w)$ is written as

$$\text{Tr } M(w) = P_w(x, y, z) + Q_w(x, y, z)\zeta,$$

then

$$Q_w = 0 \quad \text{and} \quad P_w = f_w.$$

Proof. By definition of $\mathfrak{A}, \mathfrak{B}$, we have

$$\text{Tr } \mathfrak{A} = x, \quad \text{Tr } \mathfrak{B} = y.$$

Also, direct multiplication gives

$$\mathfrak{A}\mathfrak{B} = \begin{pmatrix} z - \zeta & x\zeta - y \\ 0 & \zeta \end{pmatrix}, \quad \text{Tr}(\mathfrak{A}\mathfrak{B}) = z.$$

By the defining property of the trace polynomial, for any pair of determinant-one matrices (A, B) over any commutative coefficient ring to which the universal trace identity applies, one has

$$\text{Tr}(w(A, B)) = f_w(\text{Tr } A, \text{Tr } B, \text{Tr}(AB)).$$

This identity is a polynomial identity with integer coefficients, so it remains valid after specialization to the commutative ring R . Applying it to the above matrices gives

$$\text{Tr } M(w) = f_w(\text{Tr } \mathfrak{A}, \text{Tr } \mathfrak{B}, \text{Tr}(\mathfrak{A}\mathfrak{B})) = f_w(x, y, z).$$

The right-hand side belongs to the subring $\mathbb{Z}[x, y, z] \subset R$. Since every element of R has a unique expression $P + Q\zeta$, it follows that $Q_w = 0$ and $P_w = f_w$. $\square$

Before stating the algorithm, we spell out the representation of polynomials used in the complexity estimate. A polynomial

$$P(x, y, z) = \sum_{i+j+k \leq n} c_{ijk} x^i y^j z^k$$

of total degree at most n is stored in a *dense coefficient array*: namely, we allocate one integer entry for every triple

$$(i, j, k) \in \mathbb{Z}_{\geq 0}^3, \quad i + j + k \leq n,$$

and store the corresponding coefficient c_{ijk} , even when $c_{ijk} = 0$. Thus each such polynomial is represented by $\binom{n+3}{3} = O(n^3)$ integer entries. An element of the quotient ring R in normal form

$$P(x, y, z) + Q(x, y, z)\zeta$$

is stored as two such dense arrays, one for P and one for Q . Addition and subtraction are performed coefficient-by-coefficient, while multiplication by x, y, z , or by ζ is implemented by shifting entries and then reducing all occurrences of ζ^2 via the relation $\zeta^2 = z\zeta - 1$.

Theorem 9.4 (Polynomial-time expanded computation). *Let $w \in F(a, b)$ be given as a word of length n in the alphabet $a^{\pm 1}, b^{\pm 1}$. There is an algorithm which outputs the fully expanded Fricke polynomial*

$$f_w(x, y, z) \in \mathbb{Z}[x, y, z]$$

in deterministic Turing-machine time $O(n^5)$ and space $O(n^4)$, using dense coefficient arrays for all monomials of total degree at most n . The output has $O(n^4)$ bits.

Proof. Use the matrices $\mathfrak{A}, \mathfrak{B} \in \mathrm{SL}(2, R)$ and the letter matrices $M(a), M(a^{-1}), M(b), M(b^{-1})$ from [Proposition 9.3](#). Thus every letter in $a^{\pm 1}, b^{\pm 1}$ is represented by a 2×2 matrix whose entries are among

$$0, \pm 1, x, y, \pm \zeta, z - \zeta, \zeta - z.$$

Given $w = s_1 \cdots s_n$, we multiply the corresponding matrices

$$M(w) = M(s_1) \cdots M(s_n)$$

inside the matrix ring $M_2(R)$.

More precisely, we do not manipulate arbitrary polynomials in four independent variables x, y, z, ζ . Instead, we perform all computations in the quotient ring

$$R = \mathbb{Z}[x, y, z, \zeta] / (\zeta^2 - z\zeta + 1).$$

Thus every entry of every intermediate matrix is stored uniquely as a pair

$$(P, Q) \text{ representing } P(x, y, z) + Q(x, y, z)\zeta,$$

where $P, Q \in \mathbb{Z}[x, y, z]$. Whenever a product produces a factor ζ^2 , it is immediately reduced using

$$\zeta^2 = z\zeta - 1.$$

By [Proposition 9.3](#), the trace of the resulting matrix $M(w) = M(s_1) \cdots M(s_n)$ is exactly $f_w(x, y, z)$. Therefore the algorithm outputs the polynomial part P_w in the unique expression

$$\mathrm{Tr} M(w) = P_w(x, y, z) + Q_w(x, y, z)\zeta;$$

by [Proposition 9.3](#), $Q_w = 0$ and $P_w = f_w$.

It remains to bound the running time for computing $\mathrm{Tr} M(w)$ by performing the requisite matrix multiplications in $SL(2, R)$. After the first k letters of w have been processed, every polynomial occurring in the entries of the current matrix has total degree at most k . Therefore it is enough to store coefficient arrays indexed by monomials $x^i y^j z^\ell$ with

$$i + j + \ell \leq n.$$

There are

$$\binom{n+3}{3} = O(n^3)$$

such monomials.

Multiplication of an element $P + Q\zeta \in R$ by any of the possible generator-matrix entries is rather simple. Multiplication by x, y, z is just a shift of coefficient arrays, multiplication by ± 1 is a sign change, and multiplication by ζ is given by

$$(P + Q\zeta)\zeta = -Q + (P + zQ)\zeta.$$

Similarly,

$$(P + Q\zeta)(\zeta - z) = -zP - Q + P\zeta,$$

and

$$(P + Q\zeta)(z - \zeta) = zP + Q - P\zeta.$$

Thus one multiplication of the current 2×2 matrix by a generator matrix requires only a bounded number of additions and shifts of arrays of size $O(n^3)$. Hence each letter costs $O(n^3)$ integer arithmetic operations, and the whole word costs $O(n^4)$ such operations.

Finally, the bit lengths of the intermediate coefficients are only linear in n . Indeed, at each step the ℓ^1 -norm of the coefficient array of each entry is multiplied by at most an absolute constant: the update consists of a bounded number of additions, sign changes, and monomial shifts. Hence all intermediate coefficient absolute values are at most C^n for some universal constant C , so their binary lengths are $O(n)$.

We now translate the preceding count into an ordinary deterministic Turing-machine bound. Each letter update involves $O(n^3)$ coefficient-array entries, and each entry operation is an addition, subtraction, sign change, or copy involving integers of bit length $O(n)$. With schoolbook arithmetic bit operations, such an integer operation costs $O(n)$ time. Thus each letter costs $O(n^4)$ Turing-machine steps, and the whole word costs

$$O(n) \cdot O(n^4) = O(n^5)$$

steps. The space required to store a constant number of coefficient arrays is

$$O(n^3) \text{ coefficients} \times O(n) \text{ bits per coefficient} = O(n^4)$$

bits. The same bound also covers the final output table, since the output has at most $O(n^3)$ coefficients, each of $O(n)$ bits. This proves the stated deterministic Turing-machine time and space bounds. $\square$

Definition 9.5 (Character equivalence). For words $v, w \in F(a, b)$ we write

$$v \equiv_c w$$

if v and w are $\text{SL}(2, \mathbb{C})$ -character equivalent, that is, if

$$\text{Tr}(v(A, B)) = \text{Tr}(w(A, B))$$

for every pair $A, B \in \text{SL}(2, \mathbb{C})$.

Lemma 9.6 (Character equivalence and Fricke polynomials). *For any $v, w \in F(a, b)$,*

$$v \equiv_c w \iff f_v = f_w \text{ in } \mathbb{Z}[x, y, z].$$

Proof. If $f_v = f_w$, then for every $A, B \in \text{SL}(2, \mathbb{C})$ we have

$$\text{Tr}(v(A, B)) = f_v(\text{Tr } A, \text{Tr } B, \text{Tr}(AB)) = f_w(\text{Tr } A, \text{Tr } B, \text{Tr}(AB)) = \text{Tr}(w(A, B)),$$

so $v \equiv_c w$.

Conversely, suppose $v \equiv_c w$. Then the polynomial

$$F(x, y, z) := f_v(x, y, z) - f_w(x, y, z)$$

vanishes on every trace triple

$$(x, y, z) = (\text{Tr } A, \text{Tr } B, \text{Tr}(AB))$$

with $A, B \in \text{SL}(2, \mathbb{C})$. We claim that every point of $\mathbb{C}^3$ occurs as such a trace triple. Indeed, given arbitrary $x, y, z \in \mathbb{C}$, take

$$A = \begin{pmatrix} x & -1 \\ 1 & 0 \end{pmatrix}.$$

Choose $q, r \in \mathbb{C}$ satisfying

$$q - r = z, \quad qr = -1.$$

This is possible since r may be taken to be any root of $r^2 + zr + 1 = 0$ and then $q = r + z$. Put

$$B = \begin{pmatrix} 0 & q \\ r & y \end{pmatrix}.$$

Then $\det A = \det B = 1$, $\text{Tr } A = x$, $\text{Tr } B = y$, and direct multiplication gives

$$\text{Tr}(AB) = q - r = z.$$

Thus F vanishes on all of $\mathbb{C}^3$. Since $F \in \mathbb{Z}[x, y, z] \subset \mathbb{C}[x, y, z]$, it follows that $F = 0$ as a polynomial. Hence $f_v = f_w$. $\square$

Corollary 9.7 (Polynomial-time decision of $\text{SL}(2, \mathbb{C})$ -character equivalence). *There is an algorithm which, given two freely reduced words $v, w \in F(a, b)$ of length at most n , decides whether $v \equiv_c w$ in deterministic Turing-machine time $O(n^5)$.*

Proof. Apply the algorithm of [Theorem 9.4](#) to v and to w . This computes the fully expanded coefficient arrays of f_v and f_w in time $O(n^5)$ and space $O(n^4)$ for each word. By [Lemma 9.6](#), the words are character equivalent if and only if these two polynomials are equal.

To compare the two expanded polynomials, compare their dense coefficient arrays for all monomials $x^i y^j z^k$ with $i + j + k \leq n$. There are $O(n^3)$ such monomials, and by the coefficient-growth estimates used in [Theorem 9.4](#), all coefficients produced by the algorithm have $O(n)$ bits. Hence the coefficient-by-coefficient comparison costs $O(n^4)$ Turing-machine time, which is dominated by the $O(n^5)$ time needed to compute the two polynomials. Therefore the total deterministic running time is $O(n^5)$. $\square$

Remark 9.8. The preceding argument is not meant to optimize the exponent of the polynomial running time. Its point is that the fully expanded output has polynomial size: by $\deg f_w \leq n$ there are only $O(n^3)$ possible monomials, and the coefficient-growth estimates of [Section 7](#) give only $O(n)$ -bit coefficients. Thus the exponential growth of the coefficients as integers does not force exponential-time computation, because exponential magnitude corresponds to only linear bit length. The elementary implementation above gives the explicit bounds $O(n^5)$ time and $O(n^4)$ space on a deterministic Turing machine. By [Corollary 8.2](#), the sparse expanded output has worst-case size at least $\Omega(n^3)$ bits. The dense coefficient-table representation used in the algorithm has worst-case size at most $O(n^4)$ bits. Thus the naive method is polynomial and within a factor of at most $O(n^2)$ of the currently proved sparse-output lower bound; closing this gap is one of the open problems below.

10. OPEN PROBLEMS

We collect here several natural questions suggested by the preceding results.

Problem 10.1 (Generic cyclically reduced word degree). Let W_n be a uniformly random cyclically reduced word of length n in $F(a, b)$. Does the ratio

$$\frac{\deg f_{W_n}}{n}$$

converge in probability to a constant c ? If so, determine c .

The lower bound in [Corollary 5.12](#) gives $c \geq 2/3$ if the limit exists. Small computations suggest that the true value may be noticeably larger than $2/3$, but no theorem identifying the limiting constant is known to us.

Problem 10.2 (Fluctuations for positive words). Let W_n be a random positive word of length n , with independent letters and $\Pr(a) = p$, $\Pr(b) = 1 - p$. Prove a central limit theorem for

$$\deg f_{W_n}$$

around its mean-order value

$$(1 - p(1 - p))n.$$

In particular, in the unbiased case $p = 1/2$, prove a central limit theorem around $3n/4$.

This should be accessible because, for positive words, [Proposition 5.1](#) expresses the degree in terms of the number of cyclic transitions between a -runs and b -runs.

Problem 10.3 (A combinatorial formula for cyclically reduced words degree). Find a combinatorial formula for $\deg f_w$ for an arbitrary cyclically reduced word $w \in F(a, b)$.

In [Theorem 1.1](#) we obtained the general lower bound

$$\deg f_w \geq \|w\|_{\mathcal{A}} - s,$$

where $s = \|w\|_{syl}/2$ is the number of alternating a - and b -syllable pairs. Equality $\deg f_w = \|w\|_{\mathcal{A}} - s$ holds for positive words, but arbitrary freely reduced words may have higher-degree contributions. A first step is to characterize the words for which equality holds.